

# Theoretical Computer Science, exam of 2025.07.18

SURNAME \_\_\_\_\_ NAME \_\_\_\_\_ PERSONAL CODE \_\_\_\_\_

Total available time: 1h 30'. You can use books and paper notes during the exam, while electronic devices are prohibited. **You must write the solutions on this paper alone.** Each correct exercise is valued 11 points. Multichance students do not need to answer to Exercises 2.2, 2.3, and 3.2: bar here in that case [ ]

## Exercise 1

Consider the following grammar G:

$S \rightarrow ABc$

$A \rightarrow a$

$AB \rightarrow aAB$

$AB \rightarrow aB$

$B \rightarrow b$

$B \rightarrow bB$

1. What type is G in the Chomsky hierarchy?
2. Use a **non-grammatical** formalism with **minimal expressive power**, among those covered in class, that characterizes the language L generated by G.

## Exercise 2

Ada and Gino are planning for a family and are considering possible names for their future children. Each of them draws from a set  $N$  of names, which is **infinite and countable**, and provides a finite description of the (non-empty) set of names they like. For simplicity, assume these descriptions are expressed as algorithms  $A_i$ , where  $i \in \{\text{Ada}, \text{Gino}\}$ , written in the C programming language. These algorithms compute the corresponding functions  $f_i : N \rightarrow \{0, 1\}$ , returning 1 for a name they like and 0 otherwise.

1) Is it decidable to determine whether a given name  $n$  is liked by both of them?

2) Is it decidable to determine whether the set of names liked by both of them is **empty**?

3) After some analysis, it turns out that the descriptions of liked names provided by the two are not sufficiently compatible. Ada now wonders whether she would have better luck with a potential new partner  $x$  (capable of providing an analogous description of a non-empty set of liked names).

Is it decidable to determine whether the set of names liked by both Ada and the generic partner  $x$  is **empty**?

### Exercise 3

Consider the language

$$\mathbf{L} = \{ \mathbf{x} \cdot \mathbf{c} \cdot \mathbf{w} \cdot \mathbf{w}^{\mathbf{R}} \cdot \mathbf{c} \cdot \mathbf{h}(\mathbf{w}) \mid \mathbf{x}, \mathbf{w} \in \{\mathbf{a}, \mathbf{b}\}^+, |\mathbf{w}| < 13 \},$$

where  $\mathbf{h}$  is the homomorphism defined as  $\mathbf{h}(\mathbf{a}) = \mathbf{b}$ ,  $\mathbf{h}(\mathbf{b}) = \mathbf{a}$ .

1. Define a single-tape Turing Machine (TM) that recognizes  $\mathbf{L}$ , and evaluate its space and time complexity.
2. Define a RAM machine that recognizes  $\mathbf{L}$ , and evaluate its space and time complexity according to an appropriate cost criterion.

*space available for solutions*

# Solutions

## EX 1

- 1) Type 1, since the rules are monotonic, but not Context-free.
- 2) The language  $L$  generated by  $G$  is  $L = a^+ b^+ c$ , which is regular. However, the solution points out an even stronger result: even though the Kleene plus  $+$  is used,  $L$  is also a star-free language.

The language is expressed using monadic first-order logic (MFO) over the alphabet  $\{a, b, c\}$ .

The formula is:

$a(0)$

$\wedge$

$\forall x(a(x) \rightarrow a(x + 1) \vee b(x + 1))$

$\wedge$

$\forall x(b(x) \rightarrow b(x + 1) \vee c(x + 1))$

$\wedge$

$\forall x(c(x) \rightarrow \text{last}(x))$

$\wedge$

$\exists x(\text{last}(x) \wedge c(x))$

## EX 2

1. Decidable (and therefore also semi-decidable): The functions  $f_{\text{Ada}}$  and  $f_{\text{Gino}}$  are both computable and total, since they are implemented in C. So it is sufficient to check whether  $f_{\text{Ada}}(n) = f_{\text{Gino}}(n) = 1$ .
2. Decidable (and therefore also semi-decidable): This is a closed question (i.e., the answer is yes or no).
3. Undecidable and not even semi-decidable:  
The set  $N$  of names is infinite and countable, so we can equivalently consider the functions  $f_{\text{Ada}}$  and  $f_x$  as functions from the set of natural numbers to  $\{0,1\}$ . Consider the set  $F$  of computable functions  $f_x$  such that the preimage of 1 under  $f_x$  has at least one element in common with the preimage of 1 under  $f_{\text{Ada}}$ —that is, functions which describe sets of liked names that overlap with Ada's. This set  $F$  is non-empty (e.g., the constant function  $f(n) = 1$  clearly overlaps) and is not the set of all computable functions (e.g., the constant function  $f(n) = 0$  clearly doesn't). Therefore, Rice's Theorem applies, and the problem is undecidable.  
Moreover, the problem is not even semi-decidable, because its complement is semi-decidable: to establish whether the intersection is non-empty, one can proceed by diagonal enumeration of pairs  $\langle n, i \rangle$  (where  $n$  is an input and  $i$  is the number of execution steps), checking whether  $f_{\text{Ada}}(n) = f_x(n) = 1$  for some  $n$  within  $i$  steps. If such an  $n$  exists, it will eventually be found in a finite number of steps.

### EX 3

The language is regular, since  $x$  is regular and the following suffix has bounded length (at most  $13 \times 3 + 2$ ). Both the Turing Machine and the RAM machine can directly implement the behavior of the corresponding finite-state automaton.

In the case of the single-tape Turing Machine, both space and time complexity are linear in the length of the input string.

The appropriate cost criterion for the RAM machine is the **constant cost model**: the values stored in memory cells, as well as the number of memory cells (i.e., the maximum address used), are bounded constants. Under the unit cost criterion, the time complexity is clearly linear, while the space complexity is constant: similarly to a finite-state automaton, the RAM can encode the finite set corresponding to the suffix directly in its program.